

Thesis

Valentin Falzberger

July 2026

1 Introduction

2 Related Work

There have been different approaches to utilize previous iterations of SAM in anomaly detection, most of which are using other models or frameworks in addition. One of those approaches combined SAM together with CLIP (Contrastive Language-Image Pre-Training) [Li et al. 2024], with CLIP providing a rough segmentation and SAM using the resulting point and bounding box prompts to achieve better results.

Another approach was to combine SAM with logic anomaly detection [Peng et al. 2025]. With this, they used feature maps of images together with the object masks from SAM to create object feature maps. The objects are then matched and used to detect logical and structural anomalies.

Yang et al. 2024 used SAM, adding a Self-Perception Tuning (SPT) method, aiming to improve the models performance in the industrial anomaly domain. They first generate a coarse draft of a mask, refining it later. To give the encoder a better understanding anomalies, they also use a Visual-Relation-Aware Adapter (VRA-Adapter) to get relations of the different image regions.

While most other research, like the above mentioned, mostly focuses on the point and bounding box prompts SAM accepts, this work takes a look at how effective the new version of SAM 3 can be at zero shot anomaly detection using only text prompts.

3 Methodology

3.1 SAM-3

Segment Anything Model (SAM) 3 is a model that can segment objects in images or videos. As an input, this model takes concept prompts, in the form of short phrases or image exemplars or visual prompts like bounding boxes, masks or points.

To show the capabilities of SAM 3, the authors say:

"To advance PCS [Promptable Concept Segmentation], we build a scalable data engine that produces a high-quality dataset with 4M unique concept labels, including hard negatives, across images and videos. [...] SAM 3 *doubles the accuracy* of existing systems in both image and video PCS, and improves previous SAM capabilities on visual segmentation tasks." [Carion et al. 2025, p. 1]

While this shows a clear step up from SAM 2 already, the new successor also scans for all matching object in the media, as opposed to just one.

Probably some more in depth description of how it works, maybe use one of the architecture graphics

3.2 Prompts

For each error type, I tried to think of some fitting prompts that could catch the given errors. An important aspect when writing prompts was to include some similar ones, changing only one word, or the form of the word such as using singular or plural. One prompt was the respective name of the error type in the MVTech dataset, as that should be a fitting description. Additionally I wanted to see how well more general prompts can perform, so I also ran "damage", "damaged", "broken" and "error" for every type at first. Later on I dropped "error" as a prompt as it barely got any results at all, even compared to the other three.

3.3 Running the Model

As the first step I wrote all the prompts in a .json file containing an entry for each category and error type of the MVTech dataset. The model was then run with each prompt individually, returning predicted masks, corresponding certainties and anomaly maps. Since SAM-3 only returns anomaly maps and masks if there was an entity found with a certainty as high as the given threshold, the confidence threshold was set to 0.25, as that was a good tradeoff between filtering out noise and still including most relevant findings. In order to get a single mask and anomaly map per image, the found masks were joined together and anomaly maps were also joined, with the highest value per pixel being chosen. Since the evaluation also needs good pictures to check for false positives, for each run, the "good" folder of the corresponding category was run through the model. In the case of the good pictures, blank ground truth masks were created, to make the evaluation in the next step possible.

3.4 Evaluation

To compare the performance of each prompt, different metrics were calculated. While pixel-level AUROC and AUPRO scores were also noted, primarily the image-level AUROC scores were used to create graphics. I decided to choose

this metric, as it shows how well the model can differentiate the bad images from the bad ones, which is the main focus of this work. A deviation in the predicted masks is not as important in practice as finding errors in the first place. For the calculation, the integrated evaluation functions in Anomalib [Akçay et al. 2022] were used.

4 Results

As expected, the results varied a lot between prompts and categories, with some performing well and showing good results, and some not showing any at all. In a few cases they even performed either worse than a random classifier would or showed unexpected behavior.

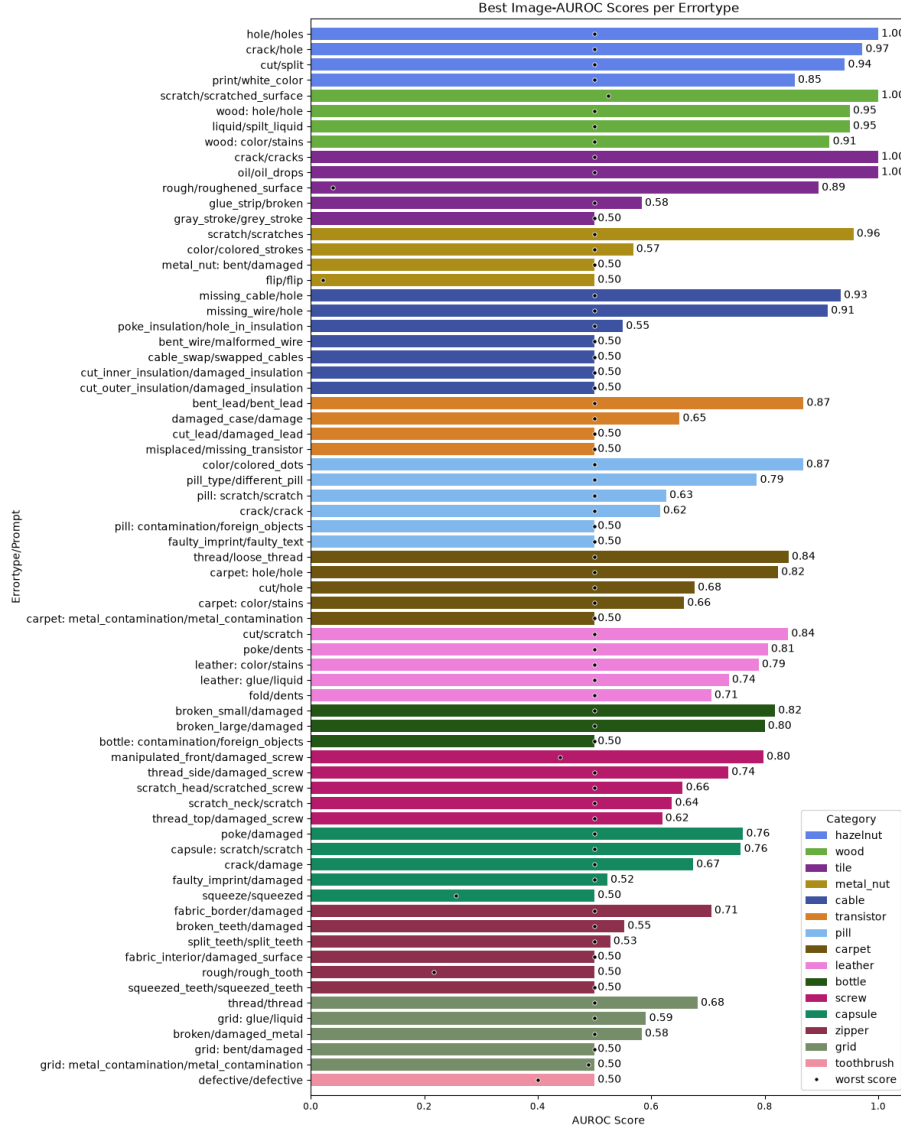


Figure 1: The best image AUROC results of each category and error type in the dataset, sorted by highest single result.

Looking at figure 1, there are four prompts that scored the best possible value of 1.0. The categories where SAM-3 performed the best overall were "hazelnut" and "wood" in which for every error type at least one prompt could score a value of 0.8 and 0.9, respectively. In the "toothbrush" category not a single prompt performed better than 0.5, meaning a random classifier would be as effective. Also "grid" and "zipper" do not have any particularly good results to show for, with most being close to 0.5. While "leather" had consistently average results, other categories such as "metal nut" or "cable" got mostly low values, with one or two relatively high ones.

As for the worst scores of each error type, most had a value of 0.5, meaning at least one prompt used could not correctly classify the images at all. However, that was not too surprising, as some prompts were intentionally vague so they could be used for all categories. Additionally, some errors are hard to accurately describe with a text prompt and some are simply not possible to find using a zero shot approach.

Interesting results are the ones with a significantly lower score than 0.5, essentially ranking good images higher than faulty ones in terms of error certainty. The lowest recorded image-AUROC score was 0.0217 in the category "metal nut", error type "flip", using the prompt "flipped nut".

category	error-type	number of prompts	average image auroc	best image auroc	best prompt
0	bottle	15	0.58	0.82	damaged
1	broken_large	5	0.61	0.80	damaged
2	broken_small	5	0.63	0.82	damaged
3	contamination	5	0.50	0.50	foreign_objects
4	cable	39	0.52	0.93	hole
5	bent_wire	5	0.50	0.50	malformed_wire
6	cable_swap	5	0.50	0.50	swapped_cables
7	cut_inner_insulation	5	0.50	0.50	damaged_insulation
8	cut_outer_insulation	5	0.50	0.50	damaged_insulation
9	missing_cable	6	0.57	0.93	hole
10	missing_wire	7	0.56	0.91	hole
11	poke_insulation	6	0.51	0.55	hole_in_insulation
12	capsule	27	0.56	0.76	damaged
13	crack	6	0.61	0.67	damage
14	faulty_imprint	5	0.51	0.52	damaged
15	poke	6	0.62	0.76	damaged
16	scratch	5	0.60	0.76	scratch
17	squeeze	5	0.45	0.50	squeezed
18	carpet	31	0.56	0.84	loose_thread
19	color	7	0.54	0.66	stains
20	cut	7	0.57	0.68	hole
21	hole	6	0.60	0.82	hole
22	metal_contamination	6	0.50	0.50	metal_contamination
23	thread	5	0.60	0.84	loose_thread
24	grid	28	0.52	0.68	thread
25	bent	6	0.50	0.50	damaged
26	broken	5	0.55	0.58	damaged_metal
27	glue	6	0.52	0.59	liquid
28	metal_contamination	6	0.50	0.50	metal_contamination
29	thread	5	0.56	0.68	thread
30	hazelnut	35	0.66	1.00	holes
31	crack	7	0.79	0.97	hole
32	cut	9	0.62	0.94	split
33	hole	8	0.74	1.00	holes
34	print	11	0.57	0.85	white_color
35	leather	30	0.59	0.84	scratch
36	color	6	0.59	0.79	stains
37	cut	6	0.62	0.84	scratch
38	fold	6	0.57	0.71	dents
39	glue	6	0.55	0.74	liquid
40	poke	6	0.60	0.81	dents
41	metal_nut	22	0.52	0.96	scratches
42	bent	5	0.50	0.50	damaged
43	color	5	0.52	0.57	colored_strokes
44	flip	5	0.40	0.50	flip
45	scratch	7	0.61	0.96	scratches
46	pill	35	0.54	0.87	colored_dots
47	color	6	0.57	0.87	colored_dots
48	contamination	6	0.50	0.50	foreign_objects
49	crack	5	0.52	0.62	crack
50	faulty_imprint	5	0.50	0.50	faulty_text
51	pill_type	6	0.61	0.79	different_pill
52	scratch	7	0.53	0.63	scratch
53	screw	27	0.54	0.80	damaged_screw
54	manipulated_front	5	0.55	0.80	damaged_screw
55	scratch_head	5	0.53	0.66	scratched_screw
56	scratch_neck	5	0.55	0.64	scratch
57	thread_side	6	0.54	0.74	damaged_screw
58	thread_top	6	0.52	0.62	damaged_screw
59	tile	31	0.60	1.00	oil_drops
60	crack	6	0.67	1.00	cracks
61	glue_strip	5	0.52	0.58	broken
62	gray_stroke	6	0.50	0.50	grey_stroke
63	oil	8	0.74	1.00	oil_drops
64	rough	6	0.49	0.89	roughened_surface
65	toothbrush	5	0.48	0.50	defective
66	defective	5	0.48	0.50	defective
67	transistor	20	0.54	0.87	bent_lead
68	bent_lead	5	0.57	0.87	bent_lead
69	cut_lead	5	0.50	0.50	damaged_lead
70	damaged_case	5	0.57	0.65	damage
71	misplaced	5	0.50	0.50	missing_transistor
72	wood	23	0.67	1.00	scratched_surface
73	color	6	0.63	0.91	stains
74	hole	6	0.71	0.95	hole
75	liquid	6	0.65	0.95	spill_liquid
76	scratch	5	0.68	1.00	scratched_surface
77	zipper	38	0.50	0.71	damaged
78	broken_teeth	8	0.51	0.55	damaged
79	fabric_border	5	0.55	0.71	damaged
80	fabric_interior	5	0.50	0.50	damaged_surface
81	rough	7	0.46	0.50	rough_tooth
82	split_teeth	7	0.50	0.53	split_teeth
83	squeezed_teeth	6	0.50	0.50	squeezed_teeth

Figure 2: Number of different prompts used per category and error type together with the best prompt for each.

4.1 Generic Prompts

After general performance, I also wanted to find out what results the generic prompts used across categories and error types yielded.

Category	Error type	Prompt	AUROC Score
hazelnut	crack	damaged	0.944444
hazelnut	crack	damage	0.888889
bottle	broken_small	damage	0.818182
bottle	broken_small	damaged	0.818182
hazelnut	hole	damaged	0.805556
bottle	broken_large	damaged	0.800000
capsule	poke	damaged	0.761905
bottle	broken_large	damage	0.750000
capsule	poke	damage	0.714286
zipper	fabric_border	damaged	0.705882
wood	hole	damaged	0.700000
hazelnut	hole	damage	0.694444
leather	cut	damaged	0.684211
capsule	crack	damage	0.673913
capsule	crack	damaged	0.652174
transistor	damaged_case	damage	0.650000
carpet	cut	damaged	0.647059
leather	cut	damage	0.631579
transistor	damaged_case	damaged	0.600000
grid	broken	damaged	0.583333
tile	glue_strip	broken	0.583333
wood	scratch	damaged	0.571429
carpet	hole	damaged	0.558824
zipper	fabric_border	damage	0.558824
hazelnut	cut	damaged	0.558824

Figure 3: The 25 error types with the best performing generic prompts.

Count of errortypes with given Image-AUROC Score						
Prompt	0.5	(0.5, 0.6]	(0.6, 0.7]	(0.7, 0.8]	(0.8, 0.9]	(0.9, 1]
broken	64	5	0	0	0	0
damage	52	9	4	2	2	0
damaged	50	8	5	2	3	1

Figure 4: Count of how often each prompt got an AUROC score within the given intervals.

Prompt	damage	damaged
Category/Errortype		
hazelnut/crack	0.888889	0.944444
bottle/broken_small	0.818182	0.818182
hazelnut/hole	0.694444	0.805556
bottle/broken_large	0.750000	0.800000
capsule/poke	0.714286	0.761905
zipper/fabric_border	0.558824	0.705882
wood/hole	0.500000	0.700000
leather/cut	0.631579	0.684211
capsule/crack	0.673913	0.652174
carpet/cut	0.529412	0.647059
transistor/damaged_case	0.650000	0.600000
grid/broken	0.500000	0.583333
wood/scratch	0.523810	0.571429
hazelnut/cut	0.529412	0.558824
carpet/hole	0.500000	0.558824
zipper/broken_teeth	0.526316	0.552632
metal_nut/color	0.500000	0.522727
capsule/faulty_imprint	0.522727	0.522727
screw/scratch_neck	0.520000	0.520000
bottle/contamination	0.500000	0.500000

Figure 5: Comparison between results of prompts "damage" and "damaged", best 20, sorted by results of "damaged".

The highest result managed to achieve a respectable AUROC score of 0.94,

as seen in Figure 3. This makes it part of the best results overall. Five more scored between 0.80 and 0.90 and across the three different prompts, 41 scored higher than 0.50. It is worth to note, that only five of those were from the prompt "broken", with the best result being 0.58, making it clearly the worst of the three.

With "damage" and "damaged" being very similar prompts I expected similar results and wanted to compare them. Looking at Figure 4, "damaged" yielded slightly better results. To accurately compare them, both AUROC scores for each error type were recorded in Figure 5. With a few exceptions, "damaged" seems to have a slightly higher or at least even score. The biggest difference was 0.20 for the error type "wood/hole", while the biggest difference in favor of "damage" was only 0.05 for "transistor/damaged_case"

4.2 Visualizations

A high image AUROC score does not always mean that the model is actually doing what we want it to. One example for this in my experiment is the error type "scratch" in the category "wood" with the prompt "scratched surface". While the calculated image AUROC score is 1.0, the highest possible value, after looking at the predicted masks, it becomes clear that the prompt does not actually find the defects I was looking for.



Figure 6: Image 000 from category "wood", error type "scratch" together with the anomaly map, predicted masks and ground truth masks, respectively. The prompt used was "scratched surface".

Here it can be clearly seen that this is not the wanted result where the model only marks the parts with scratches, but rather the whole image. In the anomaly map, the scratched parts are even given a lower confidence. This gives the impression that the prompt was not interpreted as intended and focused more on the "surface" part, ignoring the "scratched" before that.

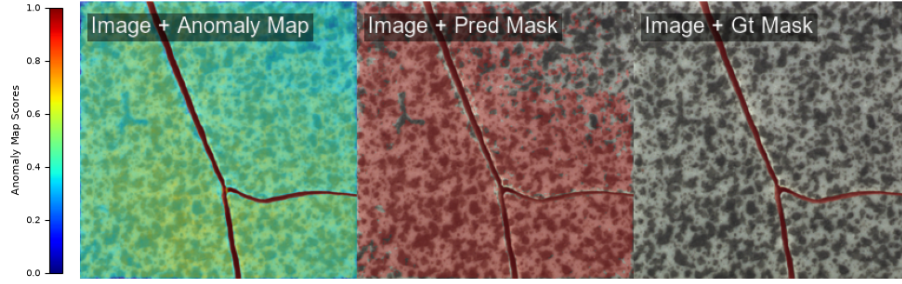


Figure 7: Image 000 from category "tile", error type "crack" together with the anomaly map, predicted masks and ground truth masks, respectively. The prompt used was "cracks".

In some cases the model correctly classified images, but also marked parts that are not faulty and do not fit the prompt. In figure 7 part of the texture is ascribed a non-zero certainty, in this case high enough to be part of the predicted mask. However, the big crack in the middle scored a much higher certainty, indicating a good performance with some noise.



Figure 8: Image 000 from category "hazelnut", error type "print" together with the anomaly map, predicted masks and ground truth masks, respectively. The prompt used was "white color".

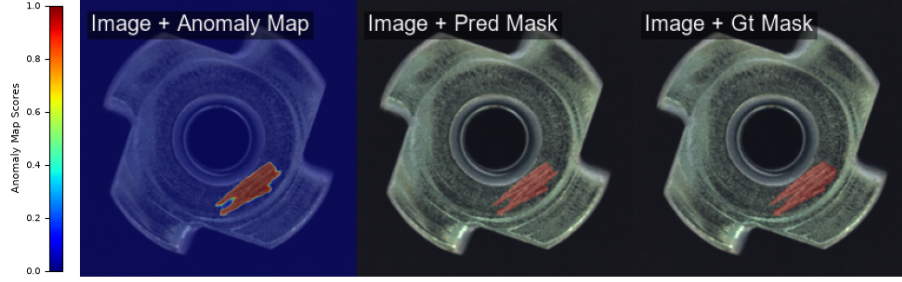


Figure 9: Image 000 from category "metal_nut", error type "scratch" together with the anomaly map, predicted masks and ground truth masks, respectively. The prompt used was "scratches".

Figure 8 and Figure 9 show visualizations of some of the better results, where the predicted mask comes very close to the ground truth mask. Across the error type "print", the pixel level AUROC score was 0.8317, for type "scratch" it was 0.8854, using the prompts "white color" and "scratches", respectively. These values are a bit lower than the corresponding image level AUROC, which is to be expected. However, as can be seen in the figures, the errors can be clearly identified from the masks.

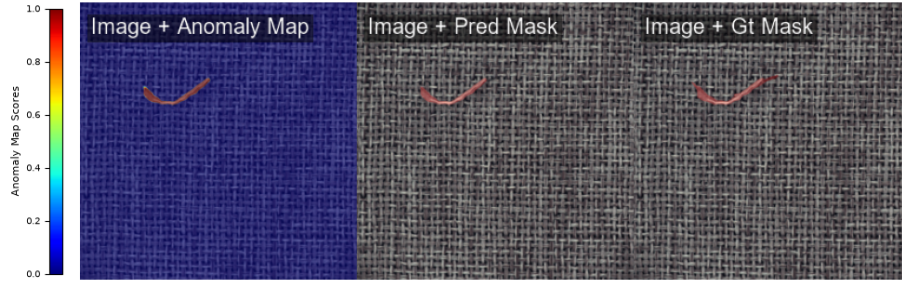


Figure 10: Image 000 from category "carpet", error type "thread" together with the anomaly map, predicted masks and ground truth masks, respectively. The prompt used was "loose thread".

While Figure 10 also shows a case of accurate predicted masks, the interesting part here is the comparison of the AUROC scores. With an image level score of 0.8421, the pixel level score is actually higher with 0.8806 for the error type "thread" using the prompt "loose thread". This means for this specific case, the model sometimes struggles to detect the error, but when it does, the masks are closer to the ground truth masks than for most other error types.

4.3 Notable Results

With the text on all the hazelnuts in the error type "print" being white, I tried out both the prompts "letters" and "white letters", assuming the latter one would perform better, or at least the same. However, it turned out to yield slightly worse results, with an image AUROC of 0.5882 for "letters" and 0.5588 for "white letters".

Maybe this part is better in discussion?

Some errors can be hard or impossible to find with zero shot anomaly detection, including error types like "misplaced_transistor", "cable_swap", "missing_cable" or "missing_wire". Without prior training on good images, the model can not know the specifics of what the product is supposed to look like and where a transistor or cable should be. There are different approaches that can work though. By looking at the consequences of the specific errors, for example a missing cable or wire leaving a hole in the space where it should be. Using this approach and prompting error types "missing_cable" and "missing_wire" with "hole" resulted in an image AUROC score of 0.9339 and 0.9112, respectively. While those are high values, this way of prompting might not be suitable for all use cases, due to making it hard to correctly classify the type of error, or some products having holes by design.

Errors like "cable_swap" could potentially also be found, by prompting for each color of cables that are needed, classifying the image as bad if one of those was not found. This is dependent on how well the model performs at finding the differently colored cables, of course. It would also need a whole different framework which would need to be tested and evaluated.

5 Discussion

Overall it can be said, that using SAM-3 with text prompts for anomaly detection can work, however it still has some clear weaknesses and shortcomings that would need to be further worked on. Judging by the results of my experiments, this can be a helpful tool in addition to other methods or an alternative if a zero shot model is needed. But also the goal of this work was not optimizing prompts for each error class, meaning there a lot of room for improvement in those with bad scores. With the top results already competing with other state-of-the-art models like PatchCore, which, using the anomalib implementation, has a mean image level AUROC score of 0.981 across the MVTec dataset, ranging from 0.934 to 1.000 per category [Akçay et al. 2022], there is a lot of potential. A possible approach to improve results could be to run multiple prompts on each image, combining the masks. This could solve the issue of hard to find catch-all prompts, making it possible to detect different types of defects.

goal wasnt to find best prompts, could be better for some categories good results

can compete with other studies, not consistent across categories though

References

- Akcay, Samet, Dick Ameln, Ashwin Vaidya, Barath Lakshmanan, Nilesch Ahuja, et al. (2022). “Anomalib: A deep learning library for anomaly detection”. In: *2022 IEEE International Conference on Image Processing (ICIP)*. IEEE, pp. 1706–1710.
- Carion, Nicolas, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, et al. (2025). *SAM 3: Segment Anything with Concepts*. arXiv: 2511.16719 [cs.CV]. URL: <https://arxiv.org/abs/2511.16719>.
- Li, Shengze, Jianjian Cao, Peng Ye, Yuhua Ding, Chongjun Tu, et al. (2024). *ClipSAM: CLIP and SAM Collaboration for Zero-Shot Anomaly Segmentation*. arXiv: 2401.12665 [cs.CV]. URL: <https://arxiv.org/abs/2401.12665>.
- Peng, Yun, Xiao Lin, Nachuan Ma, Jiayuan Du, Chuangwei Liu, et al. (2025). “SAM-LAD: Segment Anything Model meets zero-shot logic anomaly detection”. In: *Knowledge-Based Systems* 314, p. 113176. ISSN: 0950-7051. DOI: <https://doi.org/10.1016/j.knosys.2025.113176>. URL: <https://www.sciencedirect.com/science/article/pii/S0950705125002230>.
- Yang, Hui-Yue, Hui Chen, Ao Wang, Kai Chen, Zijia Lin, et al. (2024). *Promptable Anomaly Segmentation with SAM Through Self-Perception Tuning*. arXiv: 2411.17217 [cs.CV]. URL: <https://arxiv.org/abs/2411.17217>.